

End-to-End ML Versioning with Git, DVC & Google Drive

Comprehensive Technical Report | Fashion-MNIST ANN Pipeline Reproduction & Conflict Resolution

Student Author:	Android AP (abubakaramir128@gmail.com)	Target Metric:	Test Accuracy $\geq$ 85% (Achieved: 87.72%)
GitHub Repo:	https://github.com/Android1289/fashion-ann-pipeline	DVC Remote:	Google Drive (gdrive://1nezVkmPATYrsIDtxP-jrDtsCU-eYjaTf)
Frameworks:	TensorFlow 2.21, DVC 3.67, PyDrive2, Git 2.47	Dataset:	Fashion-MNIST (70,000 28x28 grayscale images, 10 classes)

Part A — Git Fundamentals & Advanced Commands

A1 & A2 (Repository Setup & Incremental Commits): The project was initialized on branch main with an initial commit containing README.md and .gitignore (configured to exclude Python caches, virtual environments, credentials, and DVC payload folders). Feature branch dev was created (git checkout -b dev) where all pipeline work was developed using 7 incremental, atomic commits representing each script and configuration component.

A3 — Git Log Variants & Explanations

- git log --oneline --graph --all:** Displays the entire commit DAG across all branches in compact single-line format with ASCII branch visualizers, revealing branch topology and merge divergences.
- git log --stat -3:** Appends diffstat summaries to the last 3 commits, revealing which files were modified and the exact line insertion/deletion count per commit.
- git log -p -1:** Shows the full line-by-line unified patch for the most recent commit, revealing exact content modifications.
- git log main..dev:** Performs symmetric difference filtering, displaying exclusively commits reachable from dev that are not in main.

```
$ git log --oneline --graph --all
* 7536b8a (HEAD -> main) Merge branch 'teammate-sim': Reconcile normalization strategies and DVC data pointers
|\
| * 44cdc83 (teammate-sim) teammate-sim: update normalization to z-score standardization
* | 6092274 main: update normalization to [-1, 1] range
|/
| * a6a07de (tag: v2, dev) Update hyperparameters (dense_units=512) and record v2 metrics
|/
* 0488933 (tag: v1) Track DVC pipeline execution v1 artifacts and metrics
* ff0360c Remove temporary utility script
* d6453fe Cleanup: Remove obsolete scratch notes using git rm
* 1a11c31 Reorganize: Move helper utility into src/ package using git mv
* 3225789 Add auxiliary utility and notes
* 80c317d Author DVC pipeline stages for end-to-end reproducibility (dvc.yaml)
* 0785fc5 Add central hyperparameters configuration (params.yaml)
* 4d37df1 Add model evaluation, confusion matrix, and metrics script (src/evaluate.py)
* ee1166c Add TensorFlow ANN model training script (src/train.py)
* 2456aa7 Add data preprocessing and validation split script (src/preprocess.py)
* fe5374f Add data preparation stage script (src/prepare.py)
* bc33b9b Initialize DVC with Google Drive remote storage
* 35561a2 Merge hotfix branch into main
|\
| * 2dce22f hotfix: Improve README description and formatting
|/
* 4e152ad Initial project setup
```

A4 — Diff Variants & Branch Comparison

- git diff:** Compares unstaged modifications in working tree against index.
- git diff --staged:** Compares indexed (staged) changes against HEAD.
- Two-Dot (main..dev) vs Three-Dot (main...dev):**
 - git diff main..dev compares current tips of both branches directly.
 - git diff main...dev computes diff between common merge base ancestor of main and dev up to dev, isolating only changes introduced on dev.

A5 to A8 — Advanced Git Scenarios (Stash, Rebase, Reset, Mv & Rm)

- A5 (Stash Scenario):** While mid-edit on src/preprocess.py, changes were stashed via git stash push -m 'WIP: Preprocessing verification edit', allowing clean context-switching to main and back to dev. git stash list verified stash persistence before restoring via git stash pop.
- A6 (Rebase Scenario):** A critical hotfix branch was branched from main, committed, and merged into main (commit 35561a2). From dev, git rebase main was executed, replaying all 7 feature commits cleanly on top of main without merge bubbles.
- A7 (Reset Scenario):** On a temporary scratch branch: git reset --soft HEAD~1 undid the commit while keeping modified files staged in the index (verified by git status). Next, git reset --hard HEAD~1 completely discarded both commit and working tree modifications, leaving repository clean.
- A8 (Reorganize with git mv & git rm):** Loose utility scripts were reorganized into src/ via git mv temp_helper.py src/utlis.py and committed as tracked rename events (100% rename similarity). Obsolete scratch notes were pruned via git rm notes_scratch.md as tracked deletion events.

Part B — Modular TensorFlow ANN Pipeline Architecture

The project decomposes the Machine Learning workflow into four single-responsibility, independent Python scripts inside src/:

Script	Input Dependencies	Produced Outputs	Role & Methodology
--------	--------------------	------------------	--------------------

src/prepare.py	Fashion-MNIST remote source	data/raw/*.npy, *.gz	Downloads and extracts rawubyte archives; saves 60,000 train and 10,000 test image/label arrays.
src/preprocess.py	data/raw/, params.yaml	data/processed/*.npy	Normalizes pixels to [0, 1] float32; performs stratified 80/20 train-validation split (48k train, 12k val).
src/train.py	data/processed/, params.yaml	models/model.h5, history.csv	Constructs Sequential ANN: Flatten(28,28) → Dense(ReLU) → Dropout → Dense(10, Softmax). Trains with Adam & Sparse Categorical Crossentropy.
src/evaluate.py	models/model.h5, data/processed/	metrics.json, confusion_matrix.png	Evaluates on test partition (10,000 images). Computes loss, accuracy, generates ConfusionMatrixDisplay.

Part C — DVC Setup with Google Drive Remote Storage

Remote Storage Architecture: DVC was initialized on feature branch dev (dvc init) and configured with Google Drive remote storage pointing to folder ID 1nezVkmPatYrslDtxP-jrDtsCU-eYjaTf:

```
dvc remote add -d gdrive_storage gdrive://1nezVkmPatYrslDtxP-jrDtsCU-eYjaTf
```

Security & Payload Isolation: .gitignore strictly excludes heavy dataset payloads (data/raw/, data/processed/), model binaries (models/), plot outputs (confusion_matrix.png), and Google OAuth client tokens (credentials.json, *.token). Git exclusively versions lightweight hash pointer files (dvc.lock) while DVC manages remote payload synchronization.

Part D — DVC Pipeline Execution & Hyperparameter Tuning (D3 vs D4)

DVC Pipeline Authoring: The four stages were wired into dvc.yaml with precise granular dependencies and parameters. Hyperparameters are centralized in params.yaml.

```
--- D3 EXECUTION (Initial Run - All Stages Execute) ---
$ dvc repro
Running stage 'prepare': > python src/prepare.py [Downloaded and saved 8 raw files]
Running stage 'preprocess': > python src/preprocess.py [Processed 60k train/val, 10k test]
Running stage 'train': > python src/train.py [Epochs: 10, dense_units: 256, val_acc: 89.20%]
Running stage 'evaluate': > python src/evaluate.py [Test Loss: 0.3408, Test Acc: 87.72%]
Updating lock file 'dvc.lock'

--- D4 EXECUTION (Hyperparameter Tuning: dense_units increased from 256 to 512) ---
$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train': > python src/train.py [Epochs: 10, dense_units: 512, val_acc: 88.44%]
Running stage 'evaluate': > python src/evaluate.py [Test Loss: 0.3587, Test Acc: 87.37%]
Updating lock file 'dvc.lock'
```

Technical Explanation of Skipped Stages (D4): In D4, DVC checked md5 hashes of stage dependencies. Stages prepare and preprocess depend on src/prepare.py, src/preprocess.py, data/raw, and params.yaml:preprocess — none of which changed. Therefore, DVC skipped them, saving significant computation. In contrast, train depends on params.yaml:train.dense_units which changed from 256 to 512, invalidating the stage lock. Since train produced a new models/model.h5, stage evaluate also automatically re-executed.

Hyperparameter & Performance Comparison: Version 1 (v1) vs Version 2 (v2)

Experiment Version	Dense Units	Dropout Rate	Learning Rate	Validation Acc	Test Accuracy	Test Loss
Version 1 (v1)	256	0.2	0.001	89.20%	87.72%	0.3408
Version 2 (v2)	512	0.2	0.001	88.44%	87.37%	0.3587

Part E — Simulated Collaboration & Simultaneous Dual Conflict Resolution

To simulate realistic team collaboration, two divergent branches were created from v1 on main, both introducing conflicting normalization algorithms and regenerating divergent preprocessed data arrays.

- **Branch teammate-sim (E1):** Implemented **Z-score standardization** $((x - \text{mean}) / \text{std})$. Re-ran dvc repro preprocess, generating processed data with hash d7169048...dir.
- **Branch main (E2):** Independently implemented **Min-Max scaling** to $[-1, 1]$ $((x / 127.5) - 1.0)$. Re-ran dvc repro preprocess, generating processed data with hash 092a23a2...dir.
- **Merge Attempt (E3):** Running git merge teammate-sim produced simultaneous Git code and DVC pointer conflicts:

```
$ git merge teammate-sim
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.
```

Simultaneous Conflict Manifestation Across Code and Data Pointers

```
--- [CODE CONFLICT: src/preprocess.py] ---
<<<<<< HEAD
# Main branch normalization: Min-Max scaling to [-1, 1] range
x_train = (x_train.astype("float32") / 127.5) - 1.0
=====
# Teammate-sim normalization: Z-score standardization
mean = np.mean(x_train, axis=(0, 1, 2), keepdims=True)
x_train = (x_train.astype("float32") - mean) / std
>>>>>> teammate-sim

--- [DVC POINTER CONFLICT: dvc.lock (stage preprocess)] ---
outs:
- path: data/processed
<<<<<< HEAD
md5: 092a23a2c39b574669a7a655fadc4d85.dir
=====
md5: d716904822b580ae72326cd16e98f50b.dir
>>>>>> teammate-sim
```

Resolution Strategy & Verification (E4 & E5):

- 1. Code Reconciliation:** In src/preprocess.py, both approaches were reconciled by selecting standard [0, 1] float32 feature normalization, optimal for sigmoid/softmax outputs and neural gradient stability.
- 2. Data Hash Reconciliation:** The conflict markers in dvc.lock were removed. dvc repro preprocess was executed to deterministically regenerate authoritative data payloads matching reconciled code, computing valid md5 eeb8cf24...dir.
- 3. Workspace Synchronization:** dvc checkout synchronized local payload caches with the active pointer.
- 4. Verification:** dvc status verified 'Data and pipelines are up to date'. The merge commit was sealed (commit 7536b8a) and dvc repro confirmed end-to-end reproducibility.

Deliverables Verification Checklist

Item / Deliverable	Requirement	Status	Reference Details
GitHub Repository	Unsquashed commit history (Parts A–E)	VERIFIED	Branches: main, dev, teammate-sim; Tags: v1, v2
ANN Accuracy	Test Accuracy ≥ 85% on Fashion-MNIST	VERIFIED	v1: 87.72% , v2: 87.37% (both exceed rubric threshold)
DVC Remote Storage	Google Drive storage registration	VERIFIED	Folder ID: 1nezVkmPAtYrslDtxP-jrDtsCU-eYjaTf
DVC Pipeline	dvc.yaml, params.yaml, dvc repro	VERIFIED	4 stages (prepare, preprocess, train, evaluate); lock generated
Git Advanced Operations	Log, diff, stash, rebase, reset, mv, rm	VERIFIED	All workflows demonstrated and verified in commit tree
Dual Conflict Resolution	Code & pointer conflict resolution	VERIFIED	Reconciled merge commit 7536b8a cleanly committed